Computers & Industrial Engineering 210 (2025) 111550

Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie

Exact approaches for parallel machine scheduling with loading and unloading servers

Alessandro Druetto [a], Andrea Grosso [b], Jully Jeunet [c,*], Fabio Salassa [d], Enrico Chiaramello [a]

[a] Dipartimento di Informatica, Università di Torino, Corso Svizzera 185, Torino 10149, Italy
[b] Dipartimento di Matematica, Università di Torino, Via Carlo Alberto 10, Torino 10123, Italy
[c] CNRS, Université Paris Dauphine, PSL Research University, CNRS UMR[7243], Lamsade, place du Maréchal de Lattre de Tassigny, Cedex 16, 75775 Paris, France
[d] Dipartimento di Ingegneria Gestionale e della Produzione (DIGEP), Politecnico di Torino, Corso Duca degli Abruzzi 24, Torino 10129, Italy

ARTICLE INFO

Keywords:
Loading and unloading servers
Parallel machines
Makespan minimisation
Exact approaches
Constraint programming

ABSTRACT

We study the non-preemptive scheduling of general jobs on identical parallel machines, where loading and unloading operations are performed by either a single shared server or by two dedicated servers. The objective is to minimise the makespan. In the literature, exact solution approaches are scarce, and limited to the two-machine case with a single server, or to regular job sets processed on any number of machines with two dedicated servers. We propose three exact solution approaches, each of which is adapted to both problem variants for any number of machines. We adapt two existing models, the time-indexed formulation (TIF) and the arc-flow formulation (FFF), and we propose a constraint programming model (CP). The performance of the models is assessed on benchmark instances with up to 100 jobs and 7 machines. Computational results showed that the FFF and CP models outperformed TIF for instances with more than 10 jobs. CP showed an outstanding performance, solving the majority of problems with 100 jobs to optimality in a short computational time. When non optimal, CP provided feasible solutions with an extremely low gap to best bound.

1. Introduction

In manufacturing, the choice between using a single server for both loading and unloading operations or two dedicated servers depends on factors such as task complexity and production volumes. In the urban furnishing industry, a single grip organ is often used to load wood panels for milling and then unload them. A single server allows for a reduction in equipment costs and space requirements. In automotive assembly lines, a robotic arm places components onto assembly stations, while another system handles the unloading of assembled engines. Separating loading and unloading tasks allows each server to operate simultaneously, reducing bottlenecks.

In this paper, we address two variants of the identical parallel machine scheduling problem with sequence-independent loading and unloading times. In both variants the objective is to minimise the makespan when scheduling a set of non-preemptive jobs $N = \{1, 2, \ldots, n\}$ on an arbitrary number $m \geq 2$ of identical parallel machines. In the first variant denoted as $P, S1|s_j, p_j, u_j|C_{\max}$, a single server handles both loading and unloading operations: each job $j \in N$ requires a setup time s_j before processing for p_j units of time, and a unit unloading time of u_j. In the second variant $P, S2|s_j, p_j, u_j|C_{\max}$, two dedicated servers are used, one for loading and one for unloading.

In the literature, the single server problem $P, S1|s_j, p_j, u_j|C_{\max}$, has been scarcely addressed. Xie et al. (2012) proposed a modified greedy algorithm to minimise the makespan, with a worst-case bound of at most twice the optimal value. Idle times were allowed after job processing and before removal. Other references investigated the worst-case performance ratios of the longest processing time (LPT) algorithm and the list scheduling (LS) algorithm. For instance, Hu et al. (2013) examined the off-line variant of the problem where the loading and unloading durations are both equal to one. Jiang, Zhang et al. (2015) studied the same problem with two machines. Jiang, Yu et al. (2015) utilised an additional online algorithm that schedules the next job based on the maximum completion time of the two machines. Elidrissi et al. (2024) were the first authors to propose exact solution approaches to the problem with 2 machines, $P2, S1|s_j, p_j, u_j|C_{\max}$, developing several mixed-integer linear programming formulations. The most effective one

* Corresponding author.
E-mail addresses: alessandro.druetto@unito.it (A. Druetto), andrea.grosso@unito.it (A. Grosso), jully.jeunet@dauphine.fr (J. Jeunet), fabio.salassa@polito.it (F. Salassa), enrico.chiaramello@unito.it (E. Chiaramello).

https://doi.org/10.1016/j.cie.2025.111550
Received 24 June 2025; Received in revised form 16 September 2025; Accepted 20 September 2025
Available online 26 September 2025
0360-8352/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

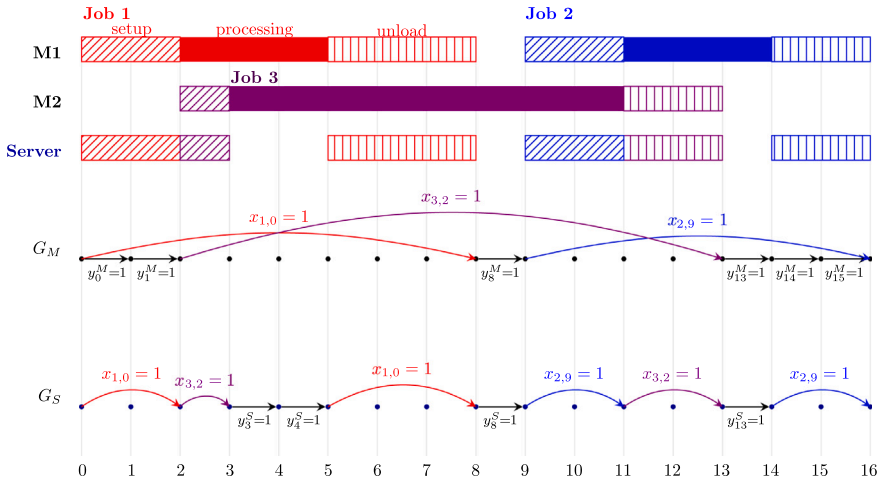

Fig. 1. A scheduling example and corresponding flows on G_M and G_S.

3.2.1. Single server for load and unload (FFF-S1)

Illustration of the problem modelling. Let us consider $n = 3$ jobs with $\{s_1, p_1, u_1\} = \{2, 3, 3\}$, $\{s_2, p_2, u_2\} = \{2, 3, 2\}$ and $\{s_3, p_3, u_3\} = \{1, 8, 2\}$. The jobs are scheduled on a time horizon $T = 16$ with one server and $m = 2$ machines. Fig. 1 displays on a same time scale the Gantt chart (upper part) of a feasible solution to the problem with $C_{\max} = 16$ and the associated flows on multigraphs G_M and G_S (lower part).

As job 1 starts at time 0 ($x_{1,0} = 1$), an execution arc is placed on the machines graph G_M and connects node 0 to node $0+s_1+p_1+u_1 = 8$. This holds even though the processing of job 1 on M1 takes place in time interval $\{2..5\}$ since machine M1 remains reserved during the setup and unloading times on the server. Thus, between 0 and 8, M1 is not idle. On the server graph, G_S, a first execution arc with a unit flow connects node 0 to node $0 + s_1 = 2$ (setup) and a second arc corresponds to the unloading phase, from $t = 5$ to $t = 8$.

As machine M1 is idle in $t = 8$ and machine M2 is idle in $t \in \{0, 1, 13, 14, 15\}$, the number of idle machines y_t^M is equal to 1 for $t \in \{0, 1, 8, 13, 14, 15\}$. This number is zero for all other points in time. The server is idle at time $\{3, 4, 8, 13\}$, hence $y_3^S = y_4^S = y_8^S = y_{13}^S = 1$. The sink node is 16 with $z_{16} = 1$.

Formulation. In order to facilitate the reading of our formulation, we index all equations with time t and we indicate in parenthesis the values of t for which the corresponding constraints hold. Our arc-flow model (FFF-S1) is written as follows

$$\min \sum_{t=1}^{T} t z_t \tag{16}$$

s.t.

$$\sum_{t=0}^{T-s_j-p_j-u_j} x_{jt} = 1 \quad (j \in N) \tag{17}$$

$$\sum_{a_{jt} \in \delta^+_{[G_M,t]}} x_{jt} + y_t^M = m \quad (t = 0) \tag{18}$$

$$\sum_{a_{jt} \in \delta^+_{[G_M,t]}} x_{jt} - \sum_{a_{j\tau} \in \delta^-_{[G_M,t]}} x_{j\tau} + y_t^M - y_{t-1}^M = -m z_t \quad (t = 1..T-1) \tag{19}$$

$$- \sum_{a_{j\tau} \in \delta^-_{[G_M,t]}} x_{j\tau} - y_{t-1}^M = -m z_t \quad (t = T) \tag{20}$$

$$\sum_{a_{jt} \in \delta^+_{[G_S,t]}} x_{jt} + y_t^S = 1 \quad (t = 0) \tag{21}$$

$$\sum_{a_{jt} \in \delta^+_{[G_S,t]}} x_{jt} - \sum_{a_{j\tau} \in \delta^-_{[G_S,t]}} x_{j\tau} + y_t^S - y_{t-1}^S = -z_t \quad (t = 1..T-1) \tag{22}$$

$$- \sum_{a_{j\tau} \in \delta^-_{[G_S,t]}} x_{j\tau} - y_{t-1}^S = -z_t \quad (t = T) \tag{23}$$

$$x_{jt} \in \{0, 1\} \quad (j \in N, t = 0..T - s_j - p_j) \tag{24}$$

$$y_t^M \geq 0, \quad y_t^S \in \{0, 1\} \quad (t = 0..T-1) \tag{25}$$

$$z_t \in \{0, 1\} \quad (t = 1..T) \tag{26}$$

The objective function in (16) represents the makespan to be minimised. Constraints (17) ensure that each job $j = 1..n$ is assigned a starting time. Constraints (18)–(20) make variables x_{jt}, y_t^M a set of flow variables that enforce the routing of m units of flow on the machines graph G_M, from source node 0 to some sink node $t \in \{1, \ldots, T\}$ for which $z_t = 1$. Let us note that summing up constraints (18)–(20) leads to $m(1 - \sum_{t=1}^{T} z_t) = 0$ ensuring that one z_t at most can be equal to 1.

Similarly, constraints (21)–(23) make variables x_{jt}, y_t^S a set of flow variables, requiring a single unit of flow to be routed on the server graph G_S from node 0 to the sink node. This guarantees that no more than one job is executed on the server at the same time. Since $\sum_{t=1}^{T} z_t = 1$ must hold in all feasible solutions, the objective function (16) correctly represents the makespan.

Compared with the formulation of a single shared server with loading operations in Druetto et al. (2025), only constraints (17) are modified, with a summation on t that ends at $T - s_j - p_j - u_j$ instead of $T - s_j - p_j$.

3.2.2. Two dedicated servers (FFF-S2)

In FFF-S1, we have two multigraphs, one for the machines, G_M, and one for the single server doing both operations, G_S. In FFF-S2, the multigraph G_S now relates to the setup server, while an additional multigraph G_U is introduced for the unloading server. We obtain the formulation FFF-S2 by adding to FFF-S1 the following constraints on the unloading server

$$\sum_{a_{jt} \in \delta^+_{[G_U,t]}} x_{jt} + y_t^U = 1 \quad (t = 0) \tag{27}$$

$$\sum_{a_{jt} \in \delta^+_{[G_U,t]}} x_{jt} - \sum_{a_{j\tau} \in \delta^-_{[G_U,t]}} x_{j\tau} + y_t^U - y_{t-1}^U = -z_t \quad (t = 1..T-1) \tag{28}$$

$$- \sum_{a_{j\tau} \in \delta^-_{[G_U,t]}} x_{j\tau} - y_{t-1}^U = -z_t \quad (t = T) \tag{29}$$

Identical job handling. As in Druetto et al. (2025), we keep a single copy of identical jobs in terms of unit execution times (s_j, p_j, u_j) and

schedule it a number of times equal to the number of copies. This procedure is applied to both FFF-S1 and FFF-S2 models.

3.3. Constraint programming (CP)

In the Constraint Programming paradigm (CP), jobs to be scheduled can be represented as the so-called *interval variables*. An interval variable models the execution of a job's task (setup, processing, unload) over a time interval of fixed size (duration of the specific task).

Resource consumption related to task execution is modelled via *cumulative functions* that, for our tasks, are `pulse` functions with unitary value; this is used to model that a certain server/machine is busy during the execution of a task.

Precedence constraints between pairs of interval variables can be specified by the `startAtStart` and `endAtEnd` operators, whose meaning is the following:

- `startAtStart`(α, β): enforces that the tasks represented by the two interval variables α and β start at the same time instant;
- `endAtEnd`(α, β): enforces that the tasks represented by the two interval variables α and β end at the same time instant.

An enumerative algorithm equipped with constraint-specific powerful propagation procedures explores the solution space efficiently in order to determine the optimal schedule.

3.3.1. Single server for load and unload (CP-S1)

The constraint programming model for the case of a single shared server handling both loading and unloading operations, CP-S1, is formulated as follows.

$$\min \max_{j=1}^{N} \{\texttt{endOf}\,(processing[j])\} \tag{30}$$

s.t.

$$\texttt{startAtStart}\,(setup[j], processing[j]) \qquad \forall j \in N \tag{31}$$

$$\texttt{endAtEnd}\,(unload[j], processing[j]) \qquad \forall j \in N \tag{32}$$

$$\sum_{j=1}^{N} \texttt{pulse}(processing[j], 1) \leq m \tag{33}$$

$$\sum_{j=1}^{N} \texttt{pulse}(setup[j]) + \sum_{j=1}^{N} \texttt{pulse}(unload[j]) \leq 1 \tag{34}$$

with interval variables

$$\begin{cases} setup[j] & \text{with range } [0, T - p_j - u_j] & \text{and size } s_j \\ processing[j] & \text{with range } [0, T] & \text{and size } s_j + p_j + u_j \\ unload[j] & \text{with range } [s_j + p_j, T] & \text{and size } u_j \end{cases} \tag{35}$$

3.3.2. Two dedicated servers (CP-S2)

The CP model for two servers differs from the previous one only by replacing (34) with the following constraints.

$$\sum_{j=1}^{N} \texttt{pulse}\,(\texttt{setup[j]}) \leq 1 \tag{36}$$

$$\sum_{j=1}^{N} \texttt{pulse}\,(\texttt{unload[j]}) \leq 1 \tag{37}$$

Here, every cumulative function for each server is considered separately.

4. Computational experiments

The MIP models (FFF and TIF) were implemented in C++ and solved using the ILOG CPLEX Callable Library (C API), version 22.1. The CP model was written in OPL using ILOG Optimisation Studio and solved with ILOG CP Optimiser, version 22.1. All computational experiments were conducted on Ubuntu Linux 22.04 under WSL2, running on a Windows 11 machine with an Intel(R) Core(TM) i7-10700 CPU @ 2.90 GHz and 16 GB of RAM.

In this Section, we first present the tuned versions of the flow-flow models and the constraint programming models (Section 4.1). Next, we describe the benchmark instances (Section 4.2). Finally, we analyse the results (Section 4.3).

4.1. Tuned versions of FFF and CP

In order to strengthen FFF and CP models, some information gathered from the available lower bounds in Eqs. (1)–(6) can be taken into account. Also, an aggressive setting of the solver's parameters can be set in order to speed up the search and obtain better solutions.

In all FFF and CP models, the following modifications apply, regardless of the number of available servers (single or double).

4.1.1. Tuned FFF (FFT)

By leveraging the fact that no integer solution exists below the lower bound values, we can state that no z_t variable can take the value of 1 for any $t < LB^*$, where LB^* denotes the best known lower bound, as given by Eqs. (1) and (2) for the single-server case, and Eqs. (3)–(6) for the double-server case. We therefore add the following constraints to the model

$$z_t = 0 \qquad \forall t = 1..\lfloor LB^* - 1 \rfloor \tag{38}$$

These constraints strengthen the continuous relaxation of both single- and double-server formulations.

We also set the following ILOG CPLEX parameters.

- `CPXPARAM_MIP_Strategy_Search = 1` (MIP dynamic search switch): apply traditional branch-and-cut strategy; disable dynamic search.
- `CPXPARAM_MIP_Strategy_Order = 1` (MIP priority order switch): use the given priority order, if one exists, for the branching decision. Arc variables x_{jt} are sorted in increasing order of priority the closer they are to the start of the sequence, i.e., their priority linearly decrease for higher t and have a maximum for $t = 0$.
- `CPXPARAM_Emphasis_MIP = ''Feasibility''` (MIP emphasis switch): this parameter controls trade-offs between speed, feasibility, optimality, and moving bounds in MIP. When this parameter is set to "Feasibility", CPLEX frequently will generate more feasible solutions as it optimises the problem, at some sacrifice in the speed to the proof of optimality.

4.1.2. Tuned CP (CPT)

We incorporate the following constraint into the CP model to enforce a minimum value for the objective function, using LB^* as the best known lower bound from Eqs. (1) and (2) for the single server case, and Eqs. (3)–(6) for the double-server case

$$\max_{j=1}^{N} \{\texttt{endOf}\,(processing[j])\} \geq LB^* \tag{39}$$

In addition, the following ILOG CP Optimiser parameters have been set.

- `SearchType = ''Restart''`: this parameter determines the type of search that is applied when solving a problem; when set to "Restart", a depth-first search that restarts from time to time is applied.
- `FailureDirectedSearch = 1`: this parameter controls the usage of failure-directed search, which assumes that there is no (better) solution or that such a solution is very hard to find; it

Table 4
Performance indicators.

opt	Number of optimal solutions found in the time limit over 10 draws for a given (n, m, α_k)
cpu	Average CPU time in seconds over the number of optimal solutions. If **opt** = 0 then **cpu** is equal to 600
int-no	Number of non-optimal integer solutions found within the time limit (600 s). If **opt** = 10 then int-no is equal to 0
gap	Average gap to best bound in % over non optimal solutions. gap is zero if all optimal solutions are found

focuses therefore on a systematic exploration of the search space, first eliminating assignments that are most likely to fail.
- `FailureDirectedSearchEmphasis = 50%`: this parameter controls the fraction of available workers the CP Optimiser uses for failure-directed search.

4.2. Benchmark instances

We used the benchmark instances generated by Elidrissi et al. (2024), available at https://github.com/Elidrissi111/P2-S1-sj-tj-Cmax. In these instances, the unit processing times p_j were drawn from a discrete uniform distribution over the interval $[10, 100]$. The loading and unloading times were defined as $s_j = \alpha_k \cdot p_j$ and $u_j = \alpha_k \cdot p_j$, respectively, where the coefficients $\{\alpha_k\}_{k=1,2,3}$ were generated from the following uniform distributions

- $\alpha_1 \sim U\,[0.01, 0.10]$,
- $\alpha_2 \sim U\,[0.10, 0.20]$,
- $\alpha_3 \sim U\,[0.10, 0.50]$.

These values $\alpha_1, \alpha_2, \alpha_2$ represent respectively low, moderate and high variability of loading/unloading times relative to processing times (see Kim & Lee, 2021).

In our study, we considered a number of machines $m \in \{2, 3, 5, 7\}$, instead of fixing $m = 2$. We also grouped instances into two size categories: moderate-sized cases with $n \in \{10, 25\}$ and larger instances with $n \in \{50, 100\}$. For each combination (n, m, α_k), ten replications were generated, resulting in a total of $4 \times 3 \times 4 \times 10 = 480$ instances to be solved per variant (single and double server).

4.3. Results

Table 8 in the Appendix summarises the exact approaches that we used, in terms of equations, lower and upper bounds.

Our tables of results in the next sections report the informations described in Table 4 for all models under the single server case, and the double server one. The runtime limit was set to 600 s.

4.3.1. Moderate-sized instances

Single server. Table 5 presents the performance of each exact model for the single server case with a number of jobs $n \in \{10, 25\}$.

For $n = 10$ jobs, the Constraint Programming models are the only ones to provide optimal solutions for all 120 problems, in approximately one second of CPU time. CP and CPT therefore outperform FFT which is the second-best model, solving 103 out of 120 instances in around 40 s. This performance is similar to that of TIF. However FFT provided non-optimal solutions with a significantly better average gap to best bound (3.90% versus 9.56% for TIF).

For $n = 25$ jobs, the performance of TIF dramatically decreased, reaching only 8 optimal solutions out of 120, in 408 s on average and non-optimal solutions having an average gap to best bound of about 37%. The flow-flow models performed much better in terms of optimality, with 49 and 58 optimal solutions for FFF and FFT respectively, in about half the computation time of TIF. However, the 30 non-optimal integer solutions found by FFT still exhibited a gap below 5%, indicating strong performance even when optimality was not proven within the time limit. Again, the best solutions were provided by CP and CPT that solved to optimality half of the instances in less than 15 s on average for CPT. Additionally both models were able to provide feasible solutions within the time limit with a very low gap to best bound of about 4%.

Double server. Table 6 provides the results when two servers are dedicated to loading and unloading operations respectively. It should first be noted that all models performed better in terms of optimality in this setting compared to the single-server case, due to the increased flexibility provided by the ability to perform loading and unloading operations concurrently. For $n = 10$ jobs, all models (except FFF) solved all 120 instances to optimality, with again an extremely low computation time of about half a second for Constraint Programming models, whereas TIF takes about 9 s on average and FFT, less than 5 s.

For $n = 25$, the performance differences among models become more pronounced. The Constraint Programming models remain the best exact approaches, solving more than 80 instances to optimality in less than 46 s on average for CPT. When non optimal, both CP models provide integer solutions with an average gap to best bound of about 8%. Flow flow models managed to solve to optimality more than a half of the instances, but required significantly more time. TIF showed a sharp decline in performance, with a quite limited number of optimal of solutions, and only under the lowest α-value. In addition, the gap to best bound for non optimal solutions was quite high, 33% on average. Overall, these results confirm that Constraint Programming models (CP and CPT) remain the most efficient for solving the double-server problem, with CPT outperforming in both optimality and computational efficiency.

4.3.2. Largest instances

On larger instances with $n \in \{50, 100\}$, only the Constraint Programming models were applied due to the inability of TIF in finding any optimal solution, and the low number of optimal solutions found by the flow flow models, within 600 s. Table 7 displays the results of CP and CPT in the single and double server cases.

In the single server situation with $n = 50$ jobs, the optimality was reached for most problems with $m \leq 3$ machines, with an average CPU time of 13.07 s for CPT, versus 30.63 s for CP. When non optimal, the best integer solutions found by both models at the time limit of 600 s exhibited a quite good average gap to best bound of about 3.5% (3.43% for CP and 3.53% for CPT). For $n = 100$, we observed a decline in the number of optimal solutions provided by CP whereas this number remains almost the same for CPT (50 optimal solutions out of 120 when $n = 50$ and 49 optimal solutions for $n = 100$). When non optimal, both models provided again extremely good average gap to best bound (2.51% for CP, and 3.20% for CPT). CPT appears more effective for a limited number of machines $m \leq 3$, while for $m \geq 3$, CP may yield non-optimal solutions but with smaller gaps to best bound.

In the double server situation, it should first be noted that CPT provided a higher number of optimal solutions than in the single-server case, and both models CP and CPT yielded non-optimal solutions with a better gap to best bound, in all instances. When $n = 50$ jobs, both models found nearly all optimal solutions for instances with $m \leq 3$ machines, with CPT achieving a higher count (64 optimal solutions versus 59 for CP), in a lower average CPU time (57.60 s versus 67.48 s on average for CP). When non optimal, the gap to best bound of CP is again slightly better (2.57% for CP and 2.79% for CPT). Under $n = 100$

Table 5
Single server — moderate-sized instances.

				TIF-S1				FFF-S1				FFT-S1				CP-S1				CPT-S1			
n	m	α_k	#	**opt**	**cpu**	int-no	gap	**opt**	**cpu**	int-no	gap	**opt**	**cpu**	int-no	gap	**opt**	**cpu**	int-no	gap	**opt**	**cpu**	int-no	gap
10	2	α_1	10	10	4.61	0	0	10	3.33	0	0	10	2.21	0	0	10	0.2	0	0	10	0.22	0	0
		α_2	10	10	12.57	0	0	10	5.97	0	0	10	3.17	0	0	10	0.38	0	0	10	0.38	0	0
		α_3	10	10	52.07	0	0	10	34.01	0	0	10	10.35	0	0	10	0.5	0	0	10	0.45	0	0
	3	α_1	10	10	2.11	0	0	10	2.18	0	0	10	1.42	0	0	10	0.71	0	0	10	0.53	0	0
		α_2	10	10	6.77	0	0	10	4.01	0	0	10	2.65	0	0	10	1.25	0	0	10	0.85	0	0
		α_3	10	5	232.94	5	12.19	0	600	10	7.62	4	237.49	6	4.57	10	3.08	0	0	10	1.84	0	0
	5	α_1	10	10	0.57	0	0	8	1.11	2	0.88	10	0.98	0	0	10	0.39	0	0	10	0.39	0	0
		α_2	10	10	12.68	0	0	10	22.09	0	0	10	3.99	0	0	10	0.74	0	0	10	0.62	0	0
		α_3	10	2	283.82	8	7.24	0	600	10	6.95	4	313.97	6	2.76	10	2.96	0	0	10	1.57	0	0
	7	α_1	10	10	0.25	0	0	10	1.24	0	0	10	0.73	0	0	10	0.18	0	0	10	0.19	0	0
		α_2	10	10	35.85	0	0	8	6.51	2	4.5	10	14.99	0	0	10	0.86	0	0	10	0.62	0	0
		α_3	10	4	286.27	6	10.45	0	600	10	6.63	5	312.09	5	4.47	10	2.72	0	0	10	1.55	0	0
Overall			**120**	**101**	**41.11**	19	9.56	**86**	**9.18**	34	6.55	**103**	**40.50**	17	3.90	**120**	**1.16**			**120**	**0.77**		
25	2	α_1	10	0	600	10	33.86	10	214.39	0	0	10	109.63	0	0	10	0.26	0	0	10	0.31	0	0
		α_2	10	0	600	10	39.56	5	431.18	4	5.16	9	258.16	0	0	10	0.5	0	0	10	0.59	0	0
		α_3	10	0	600	10	52.87	0	600	2	18.64	0	600	1	17.39	10	1.7	0	0	10	1.33	0	0
	3	α_1	10	0	600	10	31.48	10	105.82	0	0	10	75.51	0	0	10	7.54	0	0	10	2.91	0	0
		α_2	10	0	600	10	34.49	5	384.9	4	1.48	9	276.23	1	0.18	10	9.08	0	0	10	3.8	0	0
		α_3	10	0	600	10	55.86	0	600	1	29.39	0	600	0	–	0	600	10	5.17	0	600	10	5.24
	5	α_1	10	5	414.32	5	18.23	10	81.34	0	0	10	39.39	0	0	10	174.72	0	0	10	78.59	0	0
		α_2	10	0	600	10	29.13	0	600	10	15.06	0	600	10	4.55	0	600	10	6.05	0	600	10	5.06
		α_3	10	0	600	9	50.8	0	600	1	28.81	0	600	2	6.8	0	600	10	3.55	0	600	10	3.52
	7	α_1	10	3	397.74	7	5.04	9	136.39	1	0.39	10	44.04	0	0	0	600	10	3.14	0	600	10	3.23
		α_2	10	0	600	10	28.13	0	600	10	11.17	0	600	10	2.54	0	600	10	3.99	0	600	10	2.88
		α_3	10	0	600	8	51.29	0	600	0	–	0	600	4	7.77	0	600	10	3.61	0	600	10	3.45
Overall			**120**	**8**	**408.10**	109	37.14	**49**	**190.27**	33	11.56	**58**	**129.23**	28	4.76	**60**	**32.30**	60	4.25	**60**	**14.59**	60	3.90

Table 6
Double server — moderate-sized instances.

				TIF-S2				FFF-S2				FFT-S2				CP-S2				CPT-S2			
n	m	α_k	#	**opt**	**cpu**	int-no	gap	**opt**	**cpu**	int-no	gap	**opt**	**cpu**	int-no	gap	**opt**	**cpu**	int-no	gap	**opt**	**cpu**	int-no	gap
10	2	α_1	10	10	3.41	0	0	10	4.07	0	0	10	3.26	0	0	10	0.40	0	0	10	0.40	0	0
		α_2	10	10	8.40	0	0	10	6.95	0	0	10	4.67	0	0	10	0.51	0	0	10	0.45	0	0
		α_3	10	10	33.42	0	0	10	18.41	0	0	10	13.56	0	0	10	0.54	0	0	10	0.49	0	0
	3	α_1	10	10	1.77	0	0	10	1.98	0	0	10	1.53	0	0	10	0.72	0	0	10	0.54	0	0
		α_2	10	10	3.39	0	0	10	3.44	0	0	10	2.50	0	0	10	1.08	0	0	10	0.68	0	0
		α_3	10	10	21.87	0	0	10	12.38	0	0	10	8.30	0	0	10	1.23	0	0	10	0.82	0	0
	5	α_1	10	10	0.48	0	0	10	0.90	0	0	10	0.86	0	0	10	0.41	0	0	10	0.41	0	0
		α_2	10	10	2.31	0	0	10	3.36	0	0	10	1.91	0	0	10	0.51	0	0	10	0.47	0	0
		α_3	10	10	16.96	0	0	9	84.56	1	2.89	10	11.46	0	0	10	0.49	0	0	10	0.48	0	0
	7	α_1	10	10	0.23	0	0	10	0.99	0	0	10	0.59	0	0	10	0.17	0	0	10	0.18	0	0
		α_2	10	10	2.01	0	0	10	3.30	0	0	10	1.47	0	0	10	0.44	0	0	10	0.40	0	0
		α_3	10	10	14.69	0	0	9	105.91	1	1.87	10	8.86	0	0	10	0.49	0	0	10	0.40	0	0
Overall			**120**	**120**	**9.08**			**118**	**19.26**	**2**	**2.38**	**120**	**4.91**			**120**	**0.58**			**120**	**0.48**		
25	2	α_1	10	1	592.03	9	33.84	10	125.74	0	0	10	117.99	0	0	10	1.97	0	0	10	1.45	0	0
		α_2	10	0	600	10	37.61	6	317.90	4	1.14	8	396.86	2	0.90	10	1.85	0	0	10	1.62	0	0
		α_3	10	0	600	10	48.41	0	600	6	6.52	2	556.37	4	7.17	10	2.21	0	0	10	1.91	0	0
	3	α_1	10	0	600	10	31.26	10	81.80	0	0	10	90.05	0	0	10	14.36	0	0	10	5.06	0	0
		α_2	10	0	600	10	33.08	10	274.68	0	0	10	219.86	0	0	10	8.73	0	0	10	3.36	0	0
		α_3	10	0	600	10	44.97	1	450.06	8	10.56	2	539.73	3	15.26	10	12.13	0	0	10	5.08	0	0
	5	α_1	10	6	350.49	4	27.85	10	43.01	0	0	10	25.76	0	0	10	166.07	0	0	10	57.23	0	0
		α_2	10	0	600	10	18.74	9	214.36	1	4.17	10	100.83	0	0	8	317.36	2	3.74	10	157.53	0	0
		α_3	10	0	600	10	39.62	0	600	6	31.73	0	600	8	7.34	0	600	10	8.45	0	600	10	8.52
	7	α_1	10	10	177.64	0	0	10	29.51	0	0	10	19.14	0	0	0	600	10	3.7	1	455.50	9	3.85
		α_2	10	0	600	10	12.71	0	600	10	4.78	1	144.29	9	3.72	0	600	10	12.06	0	600	10	11.97
		α_3	10	0	600	10	33.46	0	600	8	19.66	0	600	10	6.27	2	199.86	8	6.58	5	233.02	5	7.51
Overall			**120**	**17**	**263.02**	**103**	**33.15**	**66**	**149**	**43**	**12.27**	**73**	**154.08**	**36**	**6.42**	**80**	**62.64**	**40**	**7.56**	**86**	**45.96**	**34**	**8.15**

Table 7
Performance of constraint programming for larger instances.

				Single server								Double server							
				CP-S1				CPT-S1				CP-S2				CPT-S2			
n	m	α_k	#	opt	cpu	int-no	gap	opt	cpu	int-no	gap	opt	cpu	int-no	gap	opt	cpu	int-no	gap
50	2	α_1	10	10	0.53	0	0	10	0.68	0	0	10	74.67	0	0	10	13.67	0	0
		α_2	10	10	1.88	0	0	10	2.34	0	0	10	17.00	0	0	10	9.34	0	0
		α_3	10	10	8.73	0	0	10	7.66	0	0	10	9.39	0	0	10	8.26	0	0
	3	α_1	10	9	67.06	1	0.10	10	27.58	0	0	9	129.80	1	0.1	10	58.55	0	0
		α_2	10	10	78.56	0	0	10	27.09	0	0	10	82.19	0	0	10	24.95	0	0
		α_3	10	0	600	10	5.61	0	600	10	5.80	10	98.09	0	0	10	37.05	0	0
	5	α_1	10	0	600	10	0.48	0	600	10	0.43	0	600	10	0.64	2	531.85	8	0.65
		α_2	10	0	600	10	6.02	0	600	10	5.92	0	600	10	1.09	2	552.38	8	1.03
		α_3	10	0	600	10	3.57	0	600	10	3.75	0	600	10	5.73	0	600	10	5.61
	7	α_1	10	0	600	10	1.61	0	600	10	1.65	0	600	10	1.56	0	600	10	1.54
		α_2	10	0	600	10	3.72	0	600	10	3.56	0	600	10	3.46	0	600	10	3.62
		α_3	10	0	600	10	3.36	0	600	10	3.60	0	600	10	3.21	0	600	10	3.52
Overall			**120**	**49**	**30.63**	71	3.43	**50**	**13.07**	70	3.53	**59**	**67.48**	61	2.57	**64**	**57.60**	56	2.79
100	2	α_1	10	10	1.13	0	0	10	1.78	0	0	6	77.78	4	0.03	10	123.97	0	0
		α_2	10	10	2.14	0	0	10	7.13	0	0	8	114.83	2	0.03	10	83.83	0	0
		α_3	10	10	87.01	0	0	10	75.33	0	0	8	77.68	2	0.06	10	66.41	0	0
	3	α_1	10	4	6.12	6	0.05	10	131.24	0	0	0	600	10	0.10	10	173.31	0	0
		α_2	10	2	95.37	8	0.06	9	259.16	1	0.04	0	600	10	0.09	8	266.51	2	0.09
		α_3	10	0	600	10	5.13	0	600	10	5.32	0	600	10	0.12	9	270.78	1	0.20
	5	α_1	10	0	600	10	0.23	0	600	10	0.27	0	600	10	0.32	0	600	10	0.32
		α_2	10	0	600	10	4.69	0	600	10	5.46	0	600	10	0.38	0	600	10	0.36
		α_3	10	0	600	10	3.49	0	600	10	3.64	0	600	10	4.96	0	600	10	4.95
	7	α_1	10	0	600	10	1.00	0	600	10	1.06	0	600	10	0.73	0	600	10	0.68
		α_2	10	0	600	10	3.25	0	600	10	3.25	0	600	10	1.24	0	600	10	1.22
		α_3	10	0	600	10	3.25	0	600	10	3.74	0	600	10	2.30	0	600	10	2.44
Overall			**120**	**36**	**31.06**	84	2.51	**49**	**91.57**	71	3.20	**22**	**91.22**	98	1.05	**57**	**158.67**	63	1.59

jobs, the number of optimal solutions found by CP drastically decreased from 59 to 22, but the gap to best bound of best integer solutions at the time limit is much lower, 1.05% on average against 2.57% when $n = 50$ jobs. CPT offers the highest number of optimal solutions (57) concentrated exclusively on the instances with a number of machines up to 3, in a reasonable CPU time of 158.67 s on average. The gap to best bound of CP is again better than that of CPT (1.05% versus 1.59%) which would suggest to utilise again CPT for instances with $m \leq 3$, and CP for instances with more than 3 machines, to benefit from near-optimal integer solutions.

The good performance of our CP models can be partly explained by the ability of ILOG CP Optimiser to efficiently handle global constraints like no-overlap and cumulative constraints, especially when interval variables are involved. ILOG CP Optimiser uses optimised propagation algorithms that help reduce the search space and speed up problem-solving. Global constraints allow solvers to process relationships between variables in a structured way rather than treating them separately, leading to more effective pruning, making such approaches particularly strong for scheduling and resource allocation problems.

5. Conclusion

In this paper, we have proposed several exact models to solve the parallel machines scheduling problem of a set of non-preemptive jobs requiring loading and unloading operations, with the aim of minimising the makespan. We considered two variants of the problem, one with a single shared server for both loading and unloading, and two dedicated servers. We have adapted two existing formulations, the time-indexed formulation (TIF) of Elidrissi et al. (2024) and the arc-flow formulation (FFF) of Druetto et al. (2025). In addition, we have proposed a new constraint programming approach (CP). The performance of the models was tested on benchmark instances generated by Elidrissi et al. (2024), with a number of jobs up to 100, and a number of machines up to 7. Computational experiments showed that TIF was unable to provide quality solutions for instances with more than 10 jobs, exhibiting a quite high gap to best bound when non optimal. The performance of the arc-flow formulation, and especially that of the tuned version (FFT) displayed a much better performance. But the CP models proved to be the most effective approach, with an outstanding performance in all situations. For the largest instances with 100 jobs, CP was able to find approximately half of the optimal solutions and when non optimal, the average gap to best bound was as low as 2%. Constraint programming therefore appears as a promising approach for solving other variants of the problem, including cases with more than two servers or sequence-dependent setup times.

CRediT authorship contribution statement

Alessandro Druetto: Writing – review & editing, Writing – original draft, Validation, Methodology. **Andrea Grosso:** Writing – review & editing, Writing – original draft, Validation, Methodology. **Jully Jeunet:** Writing – review & editing, Writing – original draft, Validation, Methodology. **Fabio Salassa:** Writing – review & editing, Writing – original draft, Validation, Methodology. **Enrico Chiaramello:** Software, Methodology.

Appendix

See Table 8.

Table 8
Summary of the exact models.

	Single server	Double-Server
Time-Indexed	TIF-S1	TIF-S2
Section	3.1.1	3.1.2
Equations	(8)–(13)	(8)–(11), (13)–(15),
Lower bound	Max. value of (1), (2)	Max. value of (3), (4), (5), (6)
Upper bound	Greedy algo. in Table 1	Greedy algo. in Table 2
Flow-Flow	FFF-S1	FFF-S2
Section	3.2.1	3.2.2
Equations	(16)–(26)	(16)–(26), (27)–(29)
Tuned	FFT	
Section	4.1.1	
Add Equation	(38)	
Lower bound	Same as TIF-S1	Same as TIF-S2
Upper bound	Same as TIF-S1	Same as TIF-S2
Constraint Prog.	CP-S1	CP-S2
Section	3.3.1	3.3.2
Equations	(30)–(35)	(30)–(33), (35)–(37)
Tuned	**CPT**	
Section	4.1.2	
Add Equation	(39)	

Data availability

Data will be made available on request.

References

Benmansour, R., & Sifaleras, A. (2021). Scheduling in parallel machines with two servers: The restrictive case. In *Lecture notes in computer science*, *Variable neighborhood search* (pp. 71–82). Springer.

Druetto, A., Grosso, A., Jeunet, J., & Salassa, F. (2025). Efficient arc-flow formulations for makespan minimisation on parallel machines with a common server. *Computers and Operations Research*, *174*, Article 106911.

Elidrissi, A., Benmansour, R., Benbrahim, M., & Duvivier, D. (2021). Mathematical formulations for the parallel machine scheduling problem with a single server. *International Journal of Production Research*, *59*(20), 6166–6184.

Elidrissi, A., Benmansour, R., Hasani, K., & Werner, F. (2024). Minimizing the makespan on two parallel machines with a common server in charge of loading and unloading operations. *Computers and Operations Research*, *167*, Article 106638.

Elidrissi, A., Benmansour, R., & Sifaleras, A. (2023). General variable neighborhood search for the parallel machine scheduling problem with two common servers. *Optimisation Letters*, *17*, 2201–2231.

Hu, J., Zhang, Q., Dong, J., & Jiang, Y. (2013). Parallel machine scheduling with a single server: loading and unloading. In P. Widmayer, Y. Xu, & B. Zhu (Eds.), *COCOA 2013, LNCS 8287* (pp. 106–116). Springer International Publishing Switzerland.

Jiang, Y., Yu, F., Zhou, P., & Hu, J. (2015). Online algorithms for scheduling two parallel machines with a single server. *International Transactions in Operational Research*, *22*, 913–927.

Jiang, Y., Zhang, Q., Hu, J., Dong, J., & Ji, M. (2015). Single-server parallel-machine scheduling with loading and unloading times. *Journal of Combinatorial Optimization*, *30*, 201–213.

Jiang, Y., Zhou, P., Wang, H., & Hu, J. (2017). Scheduling on two parallel machines with two dedicated servers. *The ANZIAM Journal*, *58*, 314–323.

Kim, M.-Y., & Lee, Y. H. (2012). MIP models and hybrid algorithm for minimizing the makespan of parallel machines scheduling problem with a single server. *Computers and Operations Research*, *39*(11), 2457–2468.

Kim, H.-J., & Lee, J.-H. (2021). Scheduling uniform parallel dedicated machines with job splitting, sequence-dependent setup times, and multiple servers. *Computers and Operations Research*, *126*, Article 105115.

Ou, J., Qi, X., & Lee, C.-Y. (2010). Parallel machine scheduling with multiple unloading servers. *Journal of Scheduling*, *13*, 213–226.

Silva, J. M. P., Subramanian, A., & Uchoa, E. (2023). On time-indexed formulations for the parallel machine scheduling problem with a common server. *Engineering Optimization*, *xx*, 1–18.

Xie, X., Zheng, Y., & Li, Y. (2012). Scheduling parallel machines with a single server: a dedicated case. In *Fifth international joint conference on computational sciences and optimization* (pp. 146–149).

Xu, D., Li, G., & Zhang, F. (2021). Scheduling an automatic IoT manufacturing system with multiple servers. *Computers & Industrial Engineering*, *157*, Article 107343.